CORE CSE SERIES — MCQ EDITION

OOP Concepts MCQ Handbook

Basic to Advanced Multiple-Choice Questions for Company OTs, PYQ-style Practice & Core Placement Preparation — with Answers & Explanations

Fundamentals | Inheritance | Polymorphism | Interfaces | SOLID & Patterns | Exceptions & Memory

Compiled for Product & Service-based Company Online Tests, GATE-style Core Practice, and Interview Rounds

Table of Contents — OOP Concepts

OOP CONCEPTS

Topic-wise MCQs with Explanations

1. OOP Fundamentals & Classes Class, object, encapsulation, abstraction

Q1 What is a 'class' in object-oriented programming? BASIC

A. A blueprint/template defining properties and behaviors that objects of that type will have

B. An instance of an object

C. A function only

D. A database table

Correct Answer: A. A blueprint/template defining properties and behaviors that objects of that type will have

Explanation: A class defines the structure (fields) and behavior (methods) that its instances (objects) will share; the object is a concrete instantiation of the class.

Q2 What is encapsulation? BASIC

A. Bundling data and methods together while restricting direct access to internal state

B. Creating multiple classes from one class

C. Allowing a method to take multiple forms

D. Converting an object to a string

Correct Answer: A. Bundling data and methods together while restricting direct access to internal state

Explanation: Encapsulation hides internal implementation details behind a controlled interface (e.g., private fields with public getters/setters), protecting object integrity.

Q3 What is abstraction in OOP? BASIC

A. Hiding complex implementation details and exposing only essential features to the user

B. Making all class members public

C. Creating a copy of an object

D. Overloading a method

Correct Answer: A. Hiding complex implementation details and exposing only essential features to the user

Explanation: Abstraction focuses on what an object does rather than how it does it, exposing a simplified interface (e.g., an abstract class or interface) while hiding internal complexity.

Q4 **Which access modifier restricts a member to be accessible only within the same class?**

BASIC

A. Private

B. Public

C. Protected

D. Default (package-private) in all languages

Correct Answer: A. Private

Explanation: A private member can only be accessed within the class it's declared in, not by subclasses or external code, enforcing strong encapsulation.

Q5 **A constructor in OOP is primarily used to:**

BASIC

A. Initialize an object's state when it is created

B. Destroy an object

C. Define static methods only

D. Compare two objects

Correct Answer: A. Initialize an object's state when it is created

Explanation: A constructor is a special method automatically invoked when an object is instantiated, typically used to initialize instance variables.

Q6 **What is a 'static' member of a class?**

INTERMEDIATE

A. A member that belongs to the class itself rather than any individual instance, shared across all objects

B. A member that can never change

C. A member only accessible in the constructor

D. A member that is always private

Correct Answer: A. A member that belongs to the class itself rather than any individual instance, shared across all objects

Explanation: Static members belong to the class as a whole (one copy shared by all instances) rather than to individual objects, and can typically be accessed without creating an instance.

2. Inheritance & Polymorphism Types of inheritance, overloading vs overriding, virtual functions

Q7 What is inheritance in OOP? BASIC

A. A mechanism where a new class derives properties and behavior from an existing class

B. A way to hide data from other classes

C. A method that can accept variable arguments

D. A type of loop

Correct Answer: A. A mechanism where a new class derives properties and behavior from an existing class

Explanation: Inheritance allows a subclass (child) to reuse and extend the fields/methods of a superclass (parent), promoting code reuse and establishing an 'is-a' relationship.

Q8 What is method overriding? BASIC

A. A subclass provides a specific implementation for a method already defined in its superclass, with the same signature

B. Defining multiple methods with the same name but different parameters in the same class

C. Calling a method from outside its class

D. Declaring a method as static

Correct Answer: A. A subclass provides a specific implementation for a method already defined in its superclass, with the same signature

Explanation: Overriding lets a subclass replace/customize inherited behavior; it requires the same method signature and is resolved at runtime (dynamic binding).

Q9 What is method overloading? BASIC

A. Defining multiple methods with the same name but different parameter lists within the same class

B. Redefining a parent class method in a child class

C. Making a method abstract

D. Declaring a method as final

Correct Answer: A. Defining multiple methods with the same name but different parameter lists within the same class

Explanation: Overloading allows multiple methods sharing a name to coexist as long as their parameter lists (type/number) differ; it's resolved at compile time.

Q10 **Runtime polymorphism in languages like Java/C++ is typically achieved through:**

INTERMEDIATE

A. Method overriding with dynamic (late) binding via virtual functions/vtables

B. Method overloading only

C. Static binding

D. Using only final classes

Correct Answer: A. Method overriding with dynamic (late) binding via virtual functions/vtables

Explanation: Runtime (dynamic) polymorphism relies on virtual method dispatch — the actual method invoked is determined at runtime based on the object's real type, not the reference type.

Q11 **Which type of inheritance involves a class inheriting from more than one base class directly?**

INTERMEDIATE

A. Multiple inheritance

B. Single inheritance

C. Hierarchical inheritance

D. Multilevel inheritance

Correct Answer: A. Multiple inheritance

Explanation: Multiple inheritance lets a class inherit from more than one parent class simultaneously; languages like Java disallow it for classes (to avoid the 'diamond problem') but allow it via interfaces.

Q12 **The 'diamond problem' in multiple inheritance refers to:**

ADVANCED

A. Ambiguity when a class inherits from two classes that both inherit from a common base class

B. A performance issue in inheritance chains

C. A syntax error in Java

D. A memory leak issue

Correct Answer: A. Ambiguity when a class inherits from two classes that both inherit from a common base class

Explanation: If class B and C both inherit from A, and class D inherits from both B and C, it's ambiguous which version of A's members D should use — this is the classic diamond problem.

Q13 **What is 'virtual function' used for in C++?**

ADVANCED

A. To enable runtime polymorphism by allowing a derived class to override a base class function

B. To make a function execute faster

C. To make a function accessible only to friend classes

D. To prevent a function from being overridden

Correct Answer: A. To enable runtime polymorphism by allowing a derived class to override a base class function

Explanation: Declaring a base class function as virtual ensures the derived class's overridden version is called via a base class pointer/reference, enabling proper runtime polymorphism.

3. Interfaces, Abstract Classes & Design Principles

Abstract vs interface, SOLID, design patterns basics

Q14 **What is the key difference between an abstract class and an interface (in languages that distinguish them)?** INTERMEDIATE

A. An abstract class can have both implemented and abstract methods plus state; an interface traditionally only declares method contracts

B. An interface can have constructors; an abstract class cannot

C. Abstract classes support multiple inheritance; interfaces do not

D. There is no meaningful difference

Correct Answer: A. An abstract class can have both implemented and abstract methods plus state; an interface traditionally only declares method contracts

Explanation: Abstract classes can hold instance state and partial implementation, while interfaces (traditionally) only define a contract of method signatures a class must implement.

Q15 **Can a class implement multiple interfaces in most modern OOP languages (e.g., Java)?** BASIC

A. Yes

B. No, only one interface is allowed

C. Only if the interfaces are empty

D. Only in abstract classes

Correct Answer: A. Yes

Explanation: Unlike class-based multiple inheritance (often restricted), a class can typically implement several interfaces, since interfaces mainly define capability contracts.

Q16 **The 'Single Responsibility Principle' (S in SOLID) states that:** INTERMEDIATE

A. A class should have only one reason to change

B. A class should implement only one interface

C. A class should have only one constructor

D. A class should be immutable

Correct Answer: A. A class should have only one reason to change

Explanation: SRP states each class/module should be responsible for a single part of functionality, making it easier to change without unintended side effects elsewhere.

Q17 **The 'Open/Closed Principle' states that software entities should be:** ADVANCED

A. Open for extension, but closed for modification

B. Open for modification, but closed for extension

C. Always open source

D. Closed to all external dependencies

Correct Answer: A. Open for extension, but closed for modification

Explanation: You should be able to add new functionality (extend behavior, e.g., via inheritance or composition) without altering existing, tested code.

Q18 **The Liskov Substitution Principle states that:** ADVANCED

A. Objects of a superclass should be replaceable with objects of a subclass without breaking the application

B. A subclass must always override every parent method

C. Interfaces must never have default methods

D. Classes must always be final

Correct Answer: A. Objects of a superclass should be replaceable with objects of a subclass without breaking the application

Explanation: LSP ensures subclasses honor the behavioral contract of their parent class, so substituting a subclass instance doesn't break correctness elsewhere in the program.

Q19 **Which design pattern ensures a class has only one instance and provides a global point of access to it?** INTERMEDIATE

A. Singleton

B. Factory

C. Observer

D. Adapter

Correct Answer: A. Singleton

Explanation: The Singleton pattern restricts instantiation of a class to a single object, typically via a private constructor and a static access method.

Q20 **The Factory design pattern is primarily used to:** INTERMEDIATE

A. Delegate object creation logic to a separate method/class rather than using 'new' directly everywhere

B. Combine two incompatible interfaces

C. Notify multiple objects of state changes

D. Add responsibilities to an object dynamically

Correct Answer: A. Delegate object creation logic to a separate method/class rather than using 'new' directly everywhere

Explanation: The Factory pattern centralizes and abstracts object-creation logic, useful when the exact type of object to create may vary based on input or configuration.

Q21 **The Observer design pattern is best suited for:** ADVANCED

A. Notifying multiple dependent objects automatically when a subject's state changes

B. Ensuring only one instance of a class exists

C. Adapting one interface to another

D. Simplifying a complex subsystem's interface

Correct Answer: A. Notifying multiple dependent objects automatically when a subject's state changes

Explanation: Observer defines a one-to-many dependency: when the subject (publisher) changes, all registered observers (subscribers) are automatically notified — the basis of pub-sub/event systems.

4. Memory, Exceptions & Advanced OOP

Constructors/destructors, exceptions, memory management

Q22 **What is a destructor used for (e.g., in C++)?** BASIC

A. To release resources and perform cleanup when an object is destroyed

B. To initialize an object

C. To copy an object

D. To compare two objects

Correct Answer: A. To release resources and perform cleanup when an object is destroyed

Explanation: A destructor is automatically invoked when an object goes out of scope or is explicitly deleted, used to free resources like memory, file handles, or network connections.

Q23 **What is 'exception handling' used for in OOP languages?** BASIC

A. To gracefully detect and respond to runtime errors without crashing the program

B. To speed up program execution

C. To define new classes dynamically

D. To handle only syntax errors

Correct Answer: A. To gracefully detect and respond to runtime errors without crashing the program

Explanation: Exception handling (try/catch/finally blocks) lets a program detect abnormal conditions at runtime and respond appropriately, rather than crashing outright.

Q24 **What is the purpose of a 'finally' block in exception handling?** INTERMEDIATE

A. To execute cleanup code regardless of whether an exception was thrown or caught

B. To catch all exception types

C. To rethrow an exception

D. To define a new exception type

Correct Answer: A. To execute cleanup code regardless of whether an exception was thrown or caught

Explanation: Code in a finally block always executes after the try/catch, whether or not an exception occurred — commonly used to release resources like file handles or connections.

Q25 **What is a 'shallow copy' of an object?** ADVANCED

A. A copy where the top-level fields are duplicated, but nested/referenced objects are still shared (same reference)

B. A copy where every nested object is also fully duplicated

C. A copy that shares no data with the original at all

D. An invalid concept in OOP

Correct Answer: A. A copy where the top-level fields are duplicated, but nested/referenced objects are still shared (same reference)

Explanation: A shallow copy duplicates the immediate object but any fields that are references to other objects still point to the SAME underlying objects as the original.

Q26 **What is a 'deep copy'?** ADVANCED

A. A copy where all nested/referenced objects are also fully and independently duplicated

B. A copy that only duplicates primitive fields

C. The same as a shallow copy

D. A copy made only at compile time

Correct Answer: A. A copy where all nested/referenced objects are also fully and independently duplicated

Explanation: A deep copy recursively duplicates every object referenced by the original, so changes to the copy never affect the original (and vice versa).

Q27 **What does 'composition over inheritance' mean as a design guideline?** ADVANCED

A. Prefer building complex objects by combining simpler ones (has-a) rather than relying heavily on class inheritance (is-a)

B. Never use inheritance under any circumstance

C. Always use interfaces instead of classes

D. Composition and inheritance are functionally identical

Correct Answer: A. Prefer building complex objects by combining simpler ones (has-a) rather than relying heavily on class inheritance (is-a)

Explanation: Composition (an object containing/using other objects to achieve behavior) is often more flexible than deep inheritance hierarchies, avoiding tight coupling and fragile base-class issues.

Q28 **What is 'operator overloading'?** INTERMEDIATE

A. Redefining how standard operators (like +, -, ==) behave for objects of a user-defined class

B. Overloading a class's constructor

C. Using an operator inside a loop

D. A synonym for method overriding

Correct Answer: A. Redefining how standard operators (like +, -, ==) behave for objects of a user-defined class

Explanation: Operator overloading (supported in languages like C++, Python) lets you define custom behavior for operators applied to instances of your own classes, e.g., adding two Vector objects with +.